



Orchestrating Streaming and Batch Data Pipelines with Airflow, Kafka, Spark, and Kubernetes on GCP

Karan Alang
August 31, 2026





Karan Alang

Principal Software Engineer

Karan Alang is a Principal Software Engineer with 25 years of experience designing scalable distributed systems, cloud-native platforms, and AI/ML solutions.

He leads the Data Engineering team at Versa Networks, where he architects real-time data pipelines and AI-powered automation frameworks.

A BCS Fellow, Senior Member of IEEE, Forbes Technology Council member, Karan is also an open-source contributor, mentor, and speaker at industry events on Data Engineering, Data Science, and AI/ML.

Returning Airflow Summit speaker — 2025: Automating threat intelligence with Airflow, XDR & LLMs

Disclaimer



The views, architectures, and opinions expressed in this presentation are solely those of the speaker in an individual capacity.

They do not represent the views or positions of any employer, organization, or client, past or present.

All architectural patterns and use cases are presented for educational purposes only.

Agenda



01 Problem Statement

High-volume, multi-tenant streaming & batch at cloud scale

02 Architecture Overview

Kafka • Spark • Airflow on GKE — two paths, one platform

03 Technology Choices

Why GKE, Managed Spark & Cloud Composer

04 Data Pipelines

Spark Structured Streaming • Batch orchestration with Airflow DAGs

05 DAG Patterns

Config-driven multi-tenant orchestration • ephemeral Dataproc jobs

06 Key Takeaways

Observability • Scalability & Cost • Security

07 Leveraging AI for Enhanced Observability

Prometheus • Gemini + ADK — AI-enhanced alerting

Problem Statement

Build a scalable, production-grade AI/ML platform on high-volume network telemetry – serving both Operational, Security use cases.

The Data Challenge

- High-volume, high-velocity syslog data from distributed network infrastructure
- Heterogeneous formats – structured, semi-structured, mixed schemas
- Zero tolerance for data loss – every event is operationally or security-relevant

The Consumer Challenge

- Operational – capacity planning & anomaly detection – tolerates batch latency
- Security – threat detection & incident response – needs near-real-time alerts
- Same raw data, radically different SLAs and output shapes

The Platform Challenge

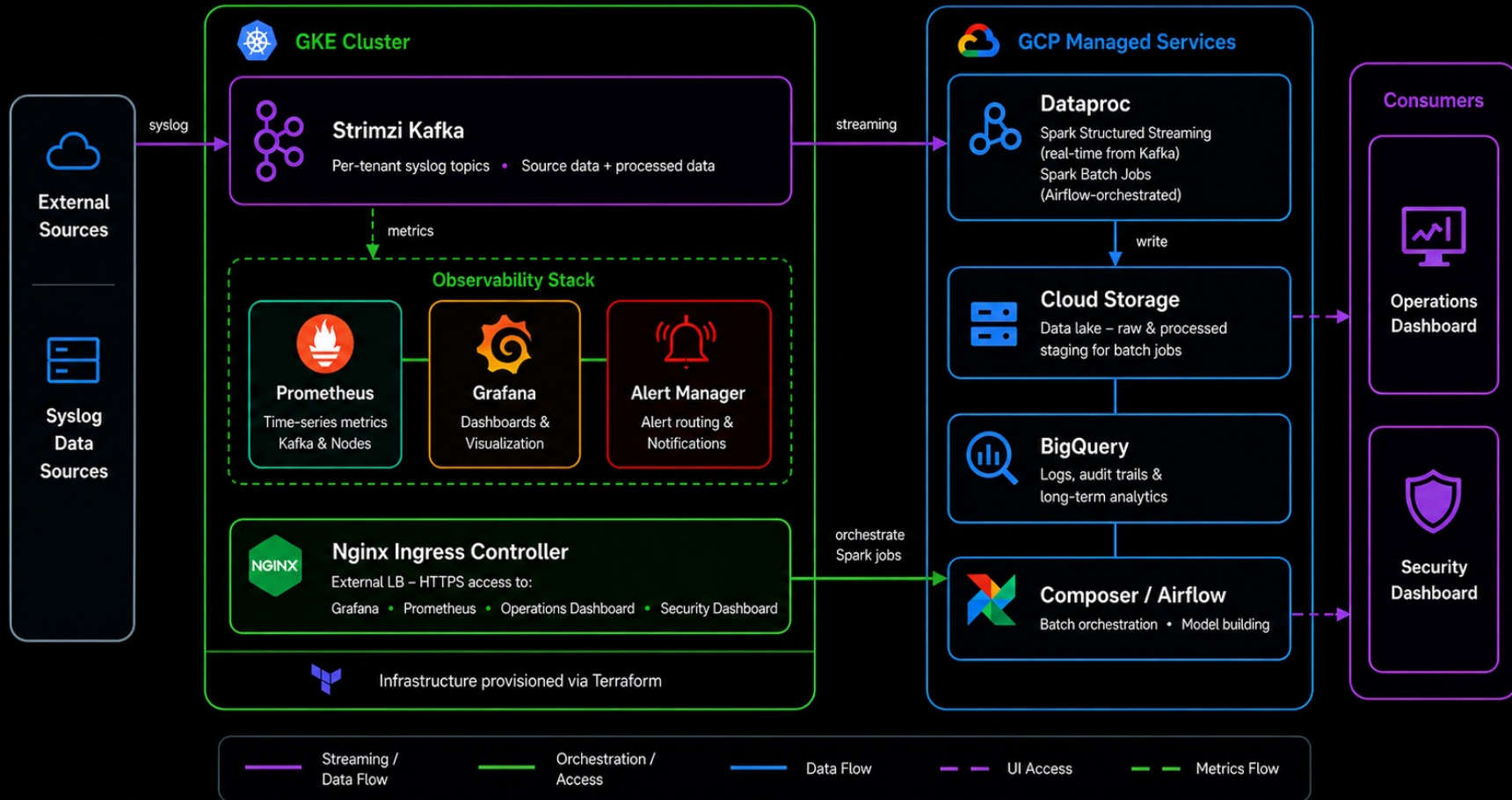
- Unified platform for streaming (real-time inference) and batch (ML training & analytics)
- Multi-tenant – one cluster, config-driven onboarding, not a code change per customer
- Elastic scale on GCP – right-sized, not over-provisioned
- Cost-effective – spot nodes, ephemeral clusters, no idle compute
- Observable, recoverable, and maintainable at production scale

The Orchestration Gap

- Streaming handles real-time ingestion – but who manages batch jobs?
- Dependency management, retries, SLA monitoring, alerting
- That gap is exactly where Airflow fits

Architecture

GCP Streaming & Batch Platform



Kubernetes/GKE – Why It's a Key Decision

Scalability & Elasticity

- Horizontal Pod Autoscaler – scales pods on CPU, memory, or custom metrics
- Node Auto-provisioner – adds capacity based on workload demand
- Streaming consumers and batch workloads scale independently

Multi-Tenant Isolation & Governance

- Namespace-level isolation – each tenant and service group separated
- Resource quotas – one tenant cannot starve others
- Network policies – fine-grained inter-service communication control
- Config-driven onboarding – new tenant = Helm install, not a code deploy

Security & Access Control

- GKE Workload Identity – pods authenticate to GCP without key files
- GCP IAM + Kubernetes RBAC – layered access control across cloud and cluster
- Vault for secret management – centralised credential store
- Audit logging – every cluster API call recorded and traceable

Infrastructure as Code – Terraform + Helm

- Terraform: cluster, node pools, namespaces, IAM – version-controlled
- Helm charts: Kafka, Airflow, Prometheus – reproducible parameterised installs
- test → production: terraform apply + helm upgrade
- No configuration drift – environment state always in source control

Observability

- Prometheus + Grafana on-cluster – unified metrics for all workloads
- AlertManager for threshold-based alerting across all pipelines
- Centralized logging – Fluentd/Loki
- Rolling deployments with health checks – zero downtime updates



Why 'Managed Apache Spark'/Dataproc on GCP?

Zero Cluster Administration

- No Spark installation, patching, or version upgrades to manage

Cost Efficiency - Autoscaling, Spot Nodes

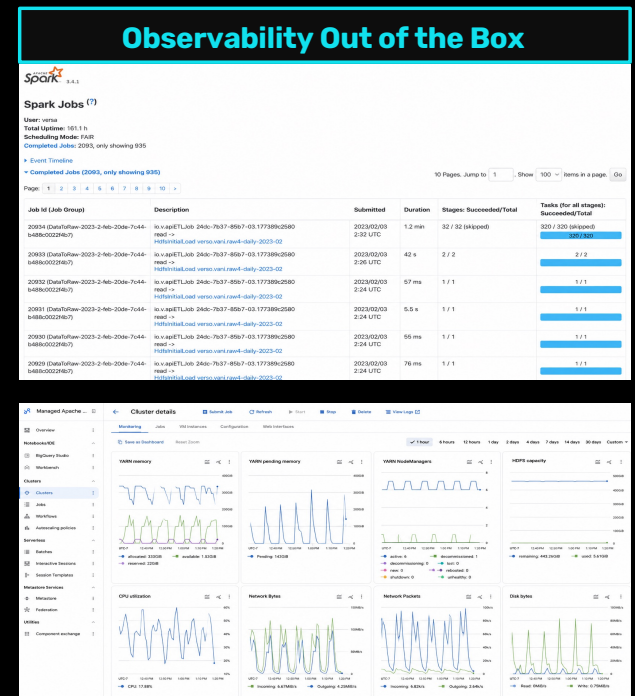
- Dataproc Autoscaling Policy – cluster nodes scale on YARN pending demand
- Spot nodes for cost-efficient burst processing

Ephemeral Clusters + Frozen Dependencies

- Clusters spin up per workload, tear down after — no always-on cost
- Pre-packaged whl archive downloaded from GCS at init — no PyPI at runtime

GCP-Native Integration

- GCS as native checkpoint and model storage – no HDFS needed
- BigQuery and Cloud Storage connectors built in – no custom integration



Multi-Tenant Streaming Architecture

- Independent Managed Apache Spark job per customer + tenant pair
- Per-customer/tenant GCS checkpoints – isolated fault domains
- New customer/tenant = config change, not a code deploy

Watermarks & State Management

- Event-time watermarks handle out-of-order data
- GCS-backed checkpoints – auto-resume from last committed offset
- Append-only output mode – no updates, no reprocessing overhead
- Bounded state prevents unbounded memory growth in long-running jobs

Security & Secret management

- Strimzi operator manages Kafka TLS – cluster CA and per-consumer client certificates auto-rotated as Kubernetes Secrets
- cert-manager + Let's Encrypt – automated TLS for externally available UIs, publicly trusted and auto-renewed
- Vault – all credentials centralized, no secrets in code or config files

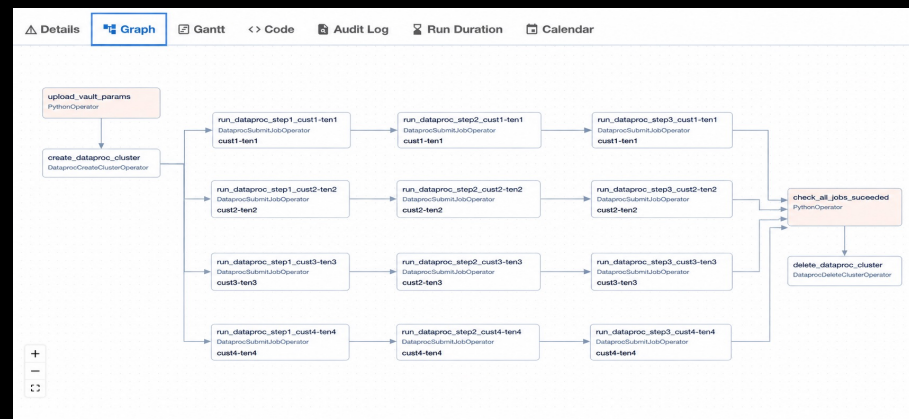
The Streaming Path

```
graph TD; A([The Streaming Path]) --> B[Multi-Tenant Streaming Architecture]; A --> C[Watermarks & State Management]; A --> D[Security & Secret management];
```

Why Cloud Composer / Airflow?

Dynamic Multi-Tenant DAG Generation

- DAG generates one Spark job per tenant from runtime config – no code changes for new tenants
- TaskGroup fans out N tenant jobs in parallel – scales without pipeline rewrites
- `max_active_runs=1` prevents overlapping DAG runs



Flexible Scheduling Across All Workloads

- Cron expressions – different cadence per DAG, per workload type
- Catchup and backfill – reprocess historical data when pipelines fail
- `schedule=None` for event-triggered runs alongside time-based ones

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions	Links
airflow_monitoring	airflow	14	* * * * *	2026-08-29, 20:20:00	2026-08-29, 20:30:00	100%	▶ ⏏	...
composer-monitor-test	airflow	4	* * * * *	2026-08-29, 20:00:00	2026-08-29, 20:30:00	100%	▶ ⏏	...
dataproc-job-monitor-test	airflow	1	* * * * *	2026-08-29, 20:00:00	2026-08-29, 20:30:00	100%	▶ ⏏	...
transform_certs_dag	versa	1	0 11 * * *	2026-08-19, 19:44:15	2026-08-31, 11:00:00	100%	▶ ⏏	...
sync-to-vault	airflow	4	None	2026-08-21, 22:33:51		100%	▶ ⏏	...



Why Cloud Composer / Airflow?

Reliability & SLA Management

- Configurable retries with backoff – transient failures handled automatically
- SLA miss alerts – operations notified before business impact
- ALL_DONE trigger – cluster teardown even on job failure

Multi-Compute Single Control Plane

- DataprocSubmitJobOperator – batch Spark on managed clusters
- KubernetesPodOperator – containerised ETL on GKE
- CloudRunExecuteJobOperator – graph-based analytics workloads

Code-Driven, Version-Controlled Pipelines

- DAGs are Python – pipeline logic in Git, reviewed like any code change
- Config-driven environment switching – test to production without code deploy
- Pre-built GCP operator ecosystem – no custom boilerplate for common operations



DAG Pattern: Config-Driven Multi-Tenant Orchestration

```
# Tenants from GCS config - add tenant = config change, not code
TENANTS = load_config(PARAMS_BUCKET)["tenants"]

with DAG("multi_tenant_pipeline",
        schedule="@hourly",
        max_active_runs=1,
        catchup=False) as dag:

    create_cluster = DataprocCreateClusterOperator(
        task_id="create_cluster", ...)

    # ALL_DONE - teardown runs even if a job fails
    delete_cluster = DataprocDeleteClusterOperator(
        task_id="delete_cluster",
        trigger_rule=TriggerRule.ALL_DONE)

    # TaskGroup fans out one Spark job per tenant in parallel
    for tenant in TENANTS:
        with TaskGroup(f"jobs_{tenant}") as tg:
            DataprocSubmitJobOperator(
                task_id=f"spark_{tenant}",
                # labels → per-tenant cost attribution in GCP billing
                job=(*build_job_config(tenant),
                    "labels": {"tenant": tenant}),
                deferrable=True)

    create_cluster >> tg >> delete_cluster
```

What This Architecture Delivers

Elastic Multi-Tenancy

- New tenants onboarded via config – zero DAG code changes
- TaskGroup parallelism scales linearly with tenant count
- One pipeline, N tenants – no rewrites as scale grows

Cost-Disciplined Compute

- Ephemeral clusters – billing tracks execution, not idle time
- ALL_DONE trigger enforces teardown even on job failure
- Label-based attribution – cost visible per tenant and workload

Production-Grade Reliability

- Self-monitoring DAGs catch silent failures before business impact
- SLA alerts fire through ops channel/email – no manual checks needed
- GCS checkpoints enable auto-resume without data loss

Key Challenges & Takeaway

Observability

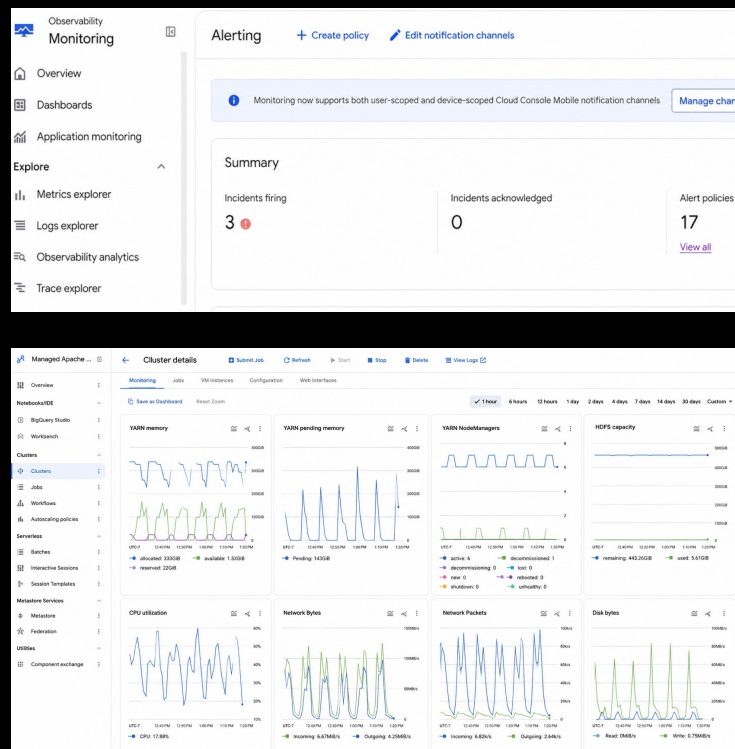
Observability is not optional, Silent failures are the most expensive kind !

Key Challenges & Takeaway

Observability

Cloud-Native Monitoring – Managed Services

- Cloud Logging + Google Alerting for Managed Airflow and Dataproc
- Alerts fire to ops channel/email on SLA miss or job failure
- Dataproc Monitoring UI for cluster stats
- Airflow UI for airflow job status



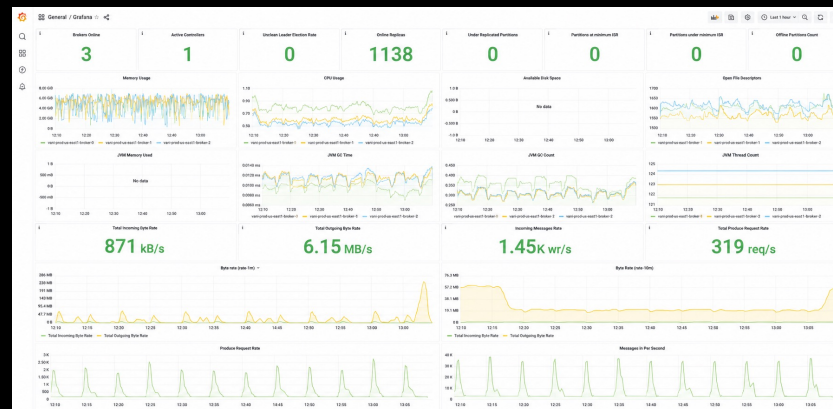
Key Challenges & Takeaway Observability

GKE Observability Stack

- Prometheus scrapes metrics from all GKE-deployed services
- Grafana dashboards — per-service, per-tenant visibility
- Alertmanager deduplication — one alert per incident, not per check

Cost-Optimised Log Management

- Export Cloud Logging to BigQuery — pay per query, not per byte stored
- Cloud Logging charges on ingestion volume; BigQuery on data scanned
- Historical log analysis at a fraction of Cloud Logging retention cost



Alertmanager Alerts Silences Status Help [New Silence](#)

Filter Group Receiver: All ☐ Silenced ☐ Inhibited

Filter by label, e.g. alertname="HighCPUUsage" [+ Silence](#)

Custom matcher, e.g. env="production"

Expand all groups

Status	Labels	Alert	Receiver	Started	Actions
▲ firing	alertname=HighCPUUsage instance=api-01 env=production +1	High CPU usage detected CPU usage is above 90% for 5m.	ops-pager	2026-08-29 20:15:42 17m ago	🔔 🔇 🔍
▲ firing	alertname=HighMemoryUsage instance=worker-03 env=production +1	High memory usage detected Memory usage is above 85% for 10m.	ops-pager	2026-08-29 20:10:11 22m ago	🔔 🔇 🔍
⏸ pending	alertname=DiskSpaceLow instance=api-01 env=production +1	Disk space low Disk usage is above 80%.	infra-team	2026-08-29 20:28:33 4m ago	🔔 🔇 🔍
✔ resolved	alertname=PodRestarting instance=web-02 env=staging +1	Pod restarting frequently Pod has restarted 3 times in 15m.	dev-team	2026-08-29 19:45:02 Ended: 2026-08-29 20:05:21	🔔 🔇 🔍

Show 25 per page

Leveraging AI for Enhanced Observability

From Alert to Diagnosis

- Prometheus detects a metric threshold breach — e.g. Kafka topic data volume spike
- Alert fires to ops channel and triggers AI investigation simultaneously
- No manual triage — AI starts correlating evidence the moment the alert fires

AI Correlates Evidence & Responds

- Reads Kafka metrics, logs, pod events, and Composer DAG state across sources
- Identifies root cause — misconfiguration, upstream event, or infrastructure issue
- Evidence-backed diagnosis posted to ops channel in 30–90 seconds

 **Observability using AI** Conversation Shared Page +

 **analysis | [Diagnosis] SyslogTopicVolumeSpike25 | warning**

 Here's my diagnosis of the `syslog.demo.cust1.ten1` alert:

Summary: The syslog topic `syslog.demo.cust1.ten1` has experienced a significant increase in message volume (over 25%) in the last hour.

Root Cause: The most likely root cause for a sudden spike in syslog volume is an external factor, such as a misconfigured network appliance, a log replay event, or a storm condition at the source. The Kafka brokers themselves are typically reactive to the incoming volume rather than the cause of the spike. However, it's prudent to ensure the brokers can handle the increased load without running out of disk space.

Evidence:

- Current Message Rate:**
 - I will check the current message rate for `syslog.demo.cust1.ten1` for the last hour to confirm the...

 **Namespace:** kafka

Key Challenges & Takeaway

Scalability & Cost

Elastic Scale on GKE

- HPA enabled — pods scale with load, no manual capacity planning
- GKE node autoscaler provisions capacity on demand
- Per-namespace quotas ensure tenant isolation under load

Ephemeral Compute for Batch

- Dataproc clusters created per run, deleted on ALL_DONE trigger
- GCP cluster autoscaling — right-sized per workload automatically
- No idle billing — cluster exists only during execution

Cost Visibility & Attribution

- Label every DAG, Streaming job, and VM — track cost per service
- GCP cost breakdown by label — identify expensive workloads immediately
- Cloud Logging can get expensive, use BigQuery or Loki for log Storage

Config-Driven Multi-Tenancy

- Add tenant = config change, not a code deploy
- TaskGroup fans out N tenant jobs in parallel — no DAG rewrites
- Dynamic job generation — architecture scales without code changes

Key Challenges & Takeaway

Security

Transport Security

- Strimzi operator manages Kafka TLS – cluster CA and per-consumer mTLS via KafkaUser, auto-rotated
- cert-manager + Let's Encrypt – automated TLS for externally available UIs, with automatic certificate renewal
- mTLS for internal service-to-service – private CA (Vault PKI) for services without public DNS

Secrets Management

- Vault – no credentials in code or config files
- GKE Workload Identity – service account bound to pod, no key files
- Secret rotation without DAG changes or redeployment

Access Control & Audit

- Cloud IAM for managed services – Composer, Dataproc, BigQuery
- Audit logging via Cloud Logging – full access trail per resource
- GKE namespace isolation – tenant workloads cannot cross boundaries



Thank You!